Computers & Security 162 (2026) 104788

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full Length Article

Vaccine: Injection vulnerabilities mitigation through dynamic process control with eBPF

Hanyu Wang [a,b], Aimin Yu [a,b], Lifang Xiao [a,*], Lixin Zhao [a], Xu Cao [a,b], Dan Meng [a,b]

[a] Institute of Information Engineerng, Chinese Academy of Sciences, Beijing, China
[b] School of CyberSecurity, University of Chinese Academy of Sciences, Beijing, China

ARTICLE INFO

Keywords:
Injection vulnerability mitigation
Dynamic process control
eBPF-based permission enforcement

ABSTRACT

Injection vulnerabilities are becoming increasingly prevalent and pose a significant threat to system security. A great deal of work highly depends on the patches for defense against these vulnerabilities. However, there is often a delay between the discovery of the vulnerabilities and the release of the corresponding patches, which leaves systems exposed to potential attacks. To address this issue, it is essential to build a vulnerability-tolerant mechanism that aims to inhibit the execution of injected payloads even when vulnerabilities are exploited. Our insight is that most of the injection vulnerabilities in operating systems can be mitigated through dynamic process control, inhibiting their ability to execute attacks. Based on this observation, we present Vaccine, a method for mitigating injection vulnerabilities by restricting attacks through dynamic process control using eBPF. Vaccine leverages process-level behavioral deviations to determine process permissions for control. By dynamically modifying process memory space to modify process permissions, Vaccine restricts the execution of injected payloads, effectively preventing attacks dynamically, hence mitigating injection vulnerabilities. The experimental results indicate that the Vaccine mitigates injection attacks exploiting 40 injection vulnerabilities. We demonstrate the behavioral deviations between processes to prove the feasibility of process-based permission control. Notably, Vaccine reduces the impact on benign behaviors by 32 % to 78 % than an advanced tool through fine-grained and dynamic control. Furthermore, it incurs low latency loss and overhead compared to three behavioral analysis methods. These results further demonstrate its practical defense against injection vulnerabilities.

1. Introduction

With the development of technology and the growth of software, the number of vulnerabilities has been steadily increasing. In 2024, a total of 39,670 CVEs were reported, marking an increase of nearly 10,000 CVEs compared to 2023 (CVE, 2024b). With the increase of vulnerabilities and their complexity, attacks become increasingly diverse, covert, and sophisticated (Tchimwa Bouom et al., 2024). Among vulnerabilities, injection vulnerabilities are ranked third in the Open Web Application Security Project (OWASP) list of the top ten most critical vulnerabilities (OWASP, 2024). They are exploited by crafting specific inputs, such as malicious code, scripts, and commands, which are mistakenly executed (Hough et al., 2020). These threats highlight the urgent necessity for effective vulnerability defense mechanisms.

More concerning is that vulnerabilities are exploited much faster than they are fixed via patches (Verizon, 2024). It leads to a time lag between the discovery of a vulnerability and the release of the patch. A great deal of work (Yang et al., 2022; Feng et al., 2024; Shao and Ding, 2024; Liu et al., 2024) contributes to detecting vulnerabilities and deploying defenses. However, they highly depend on the patches, while lacking the consideration of security measures for the time lag before the patch is available. In order to reduce system exposure to potential attacks and maintain the normal operation of applications in a vulnerable environment, methods for mitigating vulnerabilities are necessary.

We achieve vulnerability mitigation by preventing threats and narrowing the scope of attacks while accepting and tolerating the vulnerabilities, instead of entirely fixing them. We aim to establish a defense system that ensures the security of benign programs while restricting the execution and propagation of malicious ones. This solution offers innovations for vulnerability defense from the perspective of controlling attack behaviors, enhancing system resilience in the absence of patches.

* Corresponding author.
E-mail addresses: wanghanyu1999@iie.ac.cn (H. Wang), yuaimin@iie.ac.cn (A. Yu), xiaolifang@iie.ac.cn (L. Xiao), zhaolixin@iie.ac.cn (L. Zhao), steve_curcy@163.com (X. Cao), mengdan@iie.ac.cn (D. Meng).

https://doi.org/10.1016/j.cose.2025.104788
Received 19 August 2025; Received in revised form 24 October 2025; Accepted 25 November 2025
Available online 27 November 2025
0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Currently, dynamic defenses aim to invalidate attacks by continuously altering the system configurations for reducing attack surface. Traditional approaches (Zhang and Li, 2023; Tan et al., 2023a; Snow et al., 2013) often rely on global, periodic, or random changes, which introduce significant overhead and cannot implement customized defenses against specific and persistent vulnerabilities. A more sophisticated approach involves employing adaptive defense, which enables a dynamic adjustment based on the observed initial attacks and exploitation, thereby constructing intelligent and efficient defense system.

The execution of attack payloads takes various forms, prevalently with process-related attack activities, which are often described as a set of processes and their correlations and dependencies (Xu et al., 2022). In this paper, we aim to prevent injection attacks, which compromise processes by injecting and executing unauthorized code, such as unsafe input handling or memory corruption. We focus on mitigating the injection vulnerabilities that enable injection attacks by preventing the execution of attacks from injected processes. Through observation, injection attacks exploiting vulnerabilities stem from processes injected with attack payloads into their memory space. Therefore, process permission control for preventing attack behaviors is an effective method for implementing the vulnerability-tolerant mechanism. However, blindly enforcing controls leads to excessive restrictions and disrupts legitimate behaviors. To effectively mitigate injection vulnerabilities while reducing the interference with benign activities, it is essential to dynamically enforce fine-grained process control. Nevertheless, it suffers from critical limitations.

Challenge 1: Dynamic process permissions. As a crucial analysis phase for mitigating injection vulnerabilities, determining process permissions for control is hindered by two constraints. The first limitation is the multitude of processes within the system, which leads to extensive system call logs. Monitoring and analyzing these process behaviors introduces significant performance overhead and influences system efficiency. The second limitation involves the unpredictability of exploiting injection vulnerabilities. An identical injection vulnerability can result in diverse attacks with various payloads. It makes predictions of denied permissions challenging because the payloads are solely influenced by the preferences of attackers. Furthermore, the process identifier (PID) is allocated dynamically, complicating the forecasting of processes and their behaviors during each attack. To address these limitations, we selectively analyze process behaviors according to the vulnerable program location to reduce overhead. Instead of relying on precise identification of malicious processes, we focus on monitoring deviations from normal execution patterns to dynamically analyze process permissions.

Challenge 2: Fine-grained control implementation. A substantial amount of effort concerns the threat mitigation via access control. Nonetheless, they suffer from limitations regarding the granularity of control. Some studies analyze threat behaviors at the user level (Daffalla et al., 2023; Cao et al., 2021), neglecting the complexity of permission management within the same user or container. Conventional mechanisms, such as SELinux (Jaeger et al., 2003), SEAndroid (Lee et al., 2021; Pan et al., 2024), and AppArmor (Li et al., 2020), implement fine-grained control at the program level. For more fine-grained control, the security boundaries encompass every process within the operating system. However, achieving process-level control is more complex because the pertinent configuration files are locked by the kernel during process execution. Some work blocks attacks by terminating processes, and several studies (Connor et al., 2020) adopt process isolation to limit behaviors, while our work focuses on achieving fine-grained control by modifying the memory spaces of processes.

To solve the above challenges, we propose Vaccine, an injection vulnerability mitigation method through dynamic process control using eBPF along the lines of the vulnerability-tolerant mechanism. Unlike approaches that aim at precisely detecting malicious behaviors, Vaccine mitigates injection vulnerabilities by process control to inhibit injected attacks. Our approach is predicated on the insight that injection attacks require malicious process execution and can be guarded by controlling them, hence mitigating injection vulnerabilities. To achieve this, Vaccine controls the permissions of processes affected by attack payloads based on the analysis of behavioral deviations. Specifically, Vaccine extracts process behaviors related to vulnerable programs from audit logs. Then, it determines permissions of deviated processes by analyzing based on Gaussian mixture clustering (GMM) and infers behavioral deviations through Bayesian inference. These involved processes are dynamically controlled using the extended Berkeley Packet Filter (eBPF), which enforces dynamic process permission allocation based on PID. The control mechanism ensures that even if an injection vulnerability is exploited, its payload execution is restricted, preventing attacks.

We employ Vaccine in the scenarios of exploiting 40 injection vulnerabilities and generating various attacks through Proofs of Concept (PoCs) and 436 malware instances predicated on injection vulnerability exploitation. We demonstrate the effectiveness of injection vulnerability mitigation and the behavioral deviations between processes to prove the feasibility of dynamically enforcing process permission control. Afterwards, we compare the impact on legitimate behaviors by Vaccine with the related advanced tool Tetragon. Ensuring mitigation of these injection vulnerabilities, Vaccine reduces the impact on legitimate behaviors by 32% to 78% through fine-grained control. Moreover, Vaccine incurs low latency loss for permission determination, which is superior to other behavioral analysis methods. As a result, Vaccine provides a practical defense in the absence of patches.

In summary, this paper makes the following contributions:

- We propose Vaccine, an innovative injection vulnerability mitigation method based on vulnerability-tolerant mechanism. It provides a solution for defense before patch release by enforcing dynamic process permission control.
- Based on the analysis of behavioral deviations, Vaccine enforces dynamic and fine-grained control of process behaviors based on eBPF. Vaccine implement a complete defense system, which bridges the gap between behavioral analysis and dynamic control.
- We employ Vaccine in the scenarios of real-world attacks based on exploiting 40 injection vulnerabilities. It prevents injection attacks while reducing the impact on legitimate behaviors. Our results prove Vaccine mitigates them with a fine-grained and low-latency enforcement mechanism.

2. Background and related work

2.1. Process-level analysis and control

Processes constitute the fundamental execution units within the operating system. Almost all computing tasks and system operations are performed as processes. Consequently, processes serve as the primary entities for security enforcement, such as in attack scenarios of malicious code execution and unauthorized access to resources. Many works employ process-based analysis for security enforcement (Ji et al., 2017; Choraria et al., 2022; Tan et al., 2023b; Pickren et al., 2024; Zhang et al., 2022; Milajerdi et al., 2019b; Wang et al., 2020). Each process possesses a separate security context, encompassing its memory space and execution flow, which defines its permissions and interactions with system resources.

System calls serve as the interface between the user space and the kernel, enabling requests for services provided by the kernel, playing a critical role in process management. Throughout a process's lifecycle, system calls, such as *fork*, *execve*, *kill*, manage process creation, execution, and termination, while *read* and *write* enable inter-process communication. The sequence of system calls reflects process behaviors, including permission acquisition and resource access. Monitoring and analyzing these interactions provides valuable insights into potential security threats, such as abnormal program execution and illegal inter-process communication. A large amount of studies (Zhao et al., 2022; Ghavamnia et al., 2020; Gaidis et al., 2023; Xing et al., 2024; Pasquier

et al., 2017) rely on system call events for threat analysis. As critical sources for security analysis, system audit logs (Audit, 2024) chronologically record system activities, including system calls. These logs provide detailed records, enabling investigation of attack activities (Wang et al., 2020; Ding et al., 2023; Xu et al., 2022; Hassan et al., 2020), extracting anomaly patterns (Milajerdi et al., 2019a; Zeng et al., 2021; Hassan et al., 2019), and analyzing threats (Li et al., 2024; Han et al., 2020; Alsaheel et al., 2021; Skopik et al., 2023; Zhu et al., 2023).

Process control provides an effective solution for vulnerability mitigation. Many works (Yang et al., 2024; Zhang et al., 2024; Xie et al., 2022; Wu et al., 2023; Lee et al., 2023; Xu et al., 2023; Narayan et al., 2023) focus on process isolation, which restricts access between isolated areas, thereby preventing the propagation of attacks among processes. However, they prioritize memory isolation while neglecting the risks associated with process privilege abuse and system call level control. Besides, they exhibit inadequate universality and lack sufficient flexibility. Customized dynamic control is particularly essential in complex operating environments.

Unlike previous works that focus on threat detection or process isolation, Vaccine enforces dynamic and fine-grained control over processes to mitigate injection vulnerabilities. Injection vulnerabilities introduce dynamic attack vectors, leading to diverse exploits that require immediate intervention. Vaccine dynamically determines process permissions based on the analysis of behavioral deviations from benign behaviors, preventing unauthorized execution and restricting malicious behaviors while reducing the impact on legitimate behaviors.

2.2. Framework of eBPF

The Berkeley Packet Filter (BPF) facilitates the dynamic integration and execution of custom program code snippets for extending or customizing operating system kernel functionality. BPF reduces the costly context switches between kernel and user I/O, thereby improving efficiency (Jin et al., 2024). The extended BPF (eBPF) broadens capabilities and is extensively utilized for various security functions, including monitoring and tracing (Zhou et al., 2024; Yuan et al., 2022), policy enforcement (DeMarinis et al., 2020; Gaidis et al., 2023; Tian et al., 2019; Humphries et al., 2021), and complex network management (Zhong et al., 2022). These studies demonstrate the potential of eBPF in vulnerability mitigation. The execution of an eBPF program involves the following stages:

1. **Bytecode Compiler:** eBPF program defines its function in user space using kernel-native functions (helpers) to invoke various utilities and interact with kernel (Lu et al., 2024). For example, the *bpf_map_lookup_elem* helper function verifies the existence of rules that restrict access. After being processed by helpers, the data structure aligns with BPF maps, which serve as key-value pairs for storage. These programs are compiled into bytecode via LLVM or Clang tools and subsequently loaded into the kernel.
2. **Verifier:** Since eBPF programs are developed by untrusted users, the kernel employs BPF verifier to perform rigorous verification of the bytecode during loading. The verifier mainly assesses program termination, memory safety, and type safety during helper invocations.
3. **JIT Compiler:** The verified bytecode is compiled into native machine code via the in-kernel Just-In-Time (JIT) compiler to improve performance. The bytecode can be executed by the BPF interpreter without JIT compiling.
4. **Execution and Attachment:** The eBPF program is attached to kernel components according to its intended purpose. They can be integrated using mechanisms like tracepoints, Kprobes/Uprobes, and LSM hooks.

In contrast to directly modifying the kernel code, the execution of eBPF offers advantages. First, eBPF can be dynamically integrated into the kernel during runtime, without necessitating recompilation or disrupting system operation. Second, eBPF programs operate within a constrained virtual machine environment and are checked by eBPF verifier. It guarantees robust security protections against tampering or hijacking. It is proved that the verifier employs analysis, including control flow verification, type checking, data flow analysis, and bound checks for effective rejections of malicious code in eBPF at load time, including buffer overflows and infinite loops, preventing their execution (Dejaeghere et al., 2023). Besides, the operation of loading, unloading, or modifying eBPF programs is a privileged kernel operation, typically requiring capabilities such as *CAP_BPF*. Therefore, attempts to load eBPF programs with basic privileges would result in a permission denial, further enhancing its security.

2.3. Dynamic defenses and vulnerability mitigation

Dynamic defense has emerged as a critical technology for increasing adversary cost and system resilience. The core principle is to create an evolving attack surface to disrupt the attack chain. Typical methods involve moving target defense (MTD), cyber deception, and dynamic security policy adjustments. MTD (Zhang and Li, 2023; Tan et al., 2023a) increases the uncertainty and costs of attackers by proactively changing system attributes and continuously and randomly altering the system surface. For example, address space layout randomization (ASLR) (Snow et al., 2013) randomizes memory addresses to reduce the predictability of exploit payloads. Deceptions (Chen et al., 2010; Jim Solomon Raja et al., 2025) involve confusing attackers using honeypots to detect malicious activity and gather threat intelligence. Dynamic security policy adjustment (Xiao et al., 2025; Yu et al., 2021; Lee et al., 2021) adapts access control using SELinux or SEAndroid at runtime based on the current threat context. However, the traditional control mechanisms cannot achieve fine-grained process control. Focusing on injection vulnerability mitigation, the adjustment in Vaccine is precisely triggered and guided by real-time behavioral deviation analysis. Vaccine represents a shift from generalized, periodic moving targets to fine-grained, intelligently adaptive security, ensuring that defenses are directly relevant to the threat, thereby maximizing security efficacy while minimizing impact on legitimate behaviors.

Moreover, multiple isolation techniques (Narayanan et al., 2020; Loughlin et al., 2023; Herber et al., 2014) based on virtualization also contribute to blocking attack chains and mitigating vulnerabilities. They provide strong security guarantees by creating isolated execution environments with dedicated computational resources. However, their isolation unit is typically an entire operating system or container, which operates at a different defensive layer than Vaccine. Our focus is on fine-grained, intra-operating system process behavior control to mitigate injection vulnerabilities in host-side. These isolation studies are complementary to Vaccine, addressing different segments of the threat landscape.

To mitigate vulnerabilities, a large number of work is performed by fixing vulnerabilities with patches. However, there is a time lag before the release of the patch, and the vulnerable programs often require normal execution. To address this challenge, the concept of vulnerability-tolerant security mechanism was proposed in Austin et al. (2018). In the case of increasingly sophisticated attacks, it aims at constructing a fault-tolerant security system, instead of a comprehensive security mechanism. To mitigate unpatched vulnerabilities, Phoenix (Kermabon-Bobinnec et al., 2024) monitors and analyzes provenance data from vulnerable containers to identify the syscall sequences as the root cause, which is shared to protect other containers. Afterwards, it integrates Seccomp with Ptrace for runtime protection. However, Phoenix suffers from limitations because it relies on extracting uniform sequence patterns from multiple attacks, which requires consistent attack patterns that may not hold in real-world scenarios. Unlike solutions of sequence filtering that focus on container vulnerabilities, we establish benign behavioral baselines to analyze deviations and enforce process control, targeted at injection vulnerabilities on the host.

3. Motivation and problem statement

The core principle of vulnerability-tolerant is to prevent attacks exploiting vulnerabilities. In this paper, we focus on mitigating injection vulnerabilities by dynamically enforcing process control. We present a real-world composite attack scenario involving multiple steps by exploiting an injection vulnerability. Besides, we analyze the features of attacks, based on which we illustrate the sufficiency and necessity of process control instead of killing them directly. We also describe preliminary definitions and the threat model.

3.1. Motivation case: CVE-2019-9858

In the attack scenario of exploiting the CVE-2019-9858, as indicated in Fig. 1, attackers inject malicious code due to insufficient input validation checks in Horde Webmail. This attack leverages process interactions to execute malicious payloads, ultimately leading to various damages to the system.

The attack proceeds in several phases and inflicts diverse damages. Initially, the attacker scans for vulnerabilities and employs brute force on the mail system to gain access. Subsequently, the injected *apache2* process transmits the parameters of the HTTP request to PHP for parsing, which triggers the execution of a subprocess. Upon exploiting the vulnerability, it scans the system to assess network connectivity. Once verifying that the conditions are conducive for an attack, the attacker downloads and executes the attack script *eximsploit.sh*, establishing a reverse shell and executing further attacks.

This case highlights some features of process behaviors in injection attacks. Process behaviors are limited and simple that even executing a command requires spawning a subprocess. It makes it difficult for processes to mimic benign behaviors to evade detection. Besides, Fig. 1 shows that the process injected with attack payloads shows a significant behavioral deviation compared to benign processes. It is worth noting that Vaccine targets at host-based environments where process behavior patterns tends to be relatively stable and predictable. In the condition that the context, including resource utilization and network activity, remains consistent, the deviated behaviors are likely to be malicious.

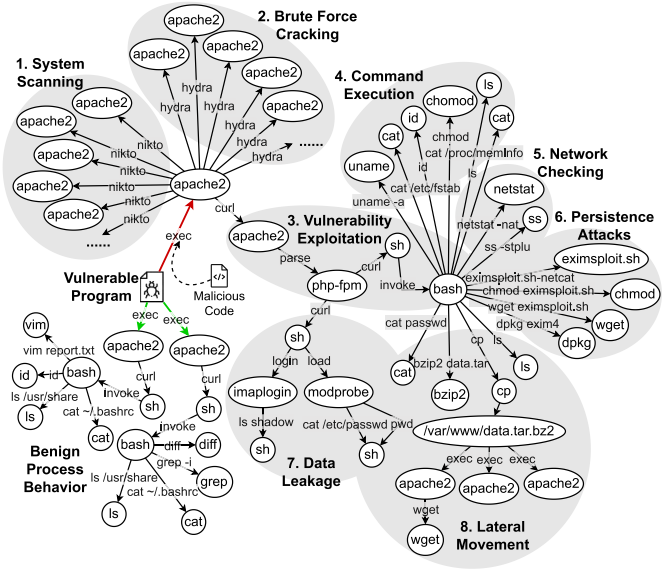


Fig. 1. The dependency graph in the scenario of exploiting CVE-2019-9858. The red line represents exploitation with deviated patterns, while green lines represent legitimate access with benign patterns. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Moreover, deviated behaviors from a vulnerable program are less likely to remain legitimate. Therefore, Vaccine determines control targets dynamically based on the behavioral deviation for control enforcement.

Besides, real-world attacks often involve multiple steps to form a progressive attack chain, such as establishing initial entry point for further lateral movement or exploiting several vulnerabilities for privilege escalation and data exfiltration. The core of Vaccine is to hinder subsequent attack steps by controlling process behavior during the initial compromise stage. On the one hand, subsequent steps in an attack chain are dependent on the successful initial exploitation. Restrictions during the initial phase also interfere with the execution of subsequent executions. Even if subsequent steps employ different techniques or attempt to trigger other vulnerabilities, their execution is trapped within the current context. On the other hand, the enforcement is not limited to the single injected process. The restrictions are propagated to the entire runtime instances interacting with the process. In this way, Vaccine effectively defends against vertical attack chains and mitigates the overall impact of vulnerabilities.

3.2. Problem statement

3.2.1. Injection attacks analysis

The injected code executes and is loaded into the memory space of a subprocess derived from the vulnerable program. In various scenarios of exploiting injection vulnerabilities, a key observation is that injection attacks typically convert from an existing process and executes within the previously legitimate context. Our control mechanism enforces security policies on potentially affected processes instead of broadly restricting all processes associated with the vulnerable program. Rather than attempting to detect all possible malicious behaviors and releases alarms, we focus on dynamically controlling processes to prevent injection attacks, hence mitigating vulnerabilities.

3.2.2. Sufficiency and necessity of fine-grained control

A key innovation of this work is the implementation of fine-grained process control rather than terminating processes directly. We analyze its feasibility and effectiveness in terms of both sufficiency and necessity.

Sufficiency. Processes are the fundamental execution units in an operating system, directly responsible for performing behaviors in the operating system. This makes process-level control an intuitive and targeted method. Furthermore, Vaccine is aimed at mitigating injection vulnerabilities on the host. Unlike network or cloud environments, which typically exhibit dynamic and diverse process execution due to virtualization and multi-tenancy, the host relies on local CPUs and operates within a constrained set of processes. Consequently, implementing process-based permission control on the host is a practical and effective approach to mitigating attacks.

Necessity. As the entities of program execution, processes are central to system behaviors. Each initiated process generally corresponds to a distinct instance of a program. Besides, programs do not automatically initiate an identical process simultaneously. Consequently, directly terminating processes potentially causes service interruptions and data loss. The issue becomes particularly critical in system environments where essential services rely on continuous process execution. Instead of employing a boolean form of defense mechanism to allow or kill processes, Vaccine dynamically restricts process capabilities to control behaviors, with the following advantages. First, legitimate processes and benign behaviors remain unaffected, which protects system availability and stability. Second, attacks are prevented at the execution level, which significantly reduces risks and limits the execution of attacks. Thus, the graceful control mechanism adopted by Vaccine ensures effectively mitigating injection attacks while maintaining system functionality.

3.2.3. Preliminaries

For precise statements, we provide explicit descriptions of behaviors and permissions. We also discuss the goal of Vaccine.

Audit Logs	System Call Behaviors
type=SYSCALL msg=audit(1582973364.663:5388095): arch=c000003e syscall=49 success=yes exit=0 a0=e a1=7ffcab80d738 a2=c a3=73 items=0 ppid=7982 pid=8610 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=4294967295 comm="auth" exe="/usr/lib/dovecot/auth" key=(null)	apache2 (7982) —bind→ auth (8610)
type=SYSCALL msg=audit(1582973364.663:5388108): arch=c000003e syscall=59 success=yes exit=0 a0=562fe6913bb8 a1=562fe6913ce0 a2=562fe6938a10 a3=73 items=2 ppid=7982 pid=8699 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=4294967295 comm="imap" exe="/usr/lib/dovecot/imap" key=(null)	apache2 (7982) —execve→ imap (8699)
type=SYSCALL msg=audit(1583350370.202:45582188): arch=c000003e syscall=59 success=yes exit=0 a0=7f2f85aadc19 a1=7fff03048920 a2=5637e99a1e20 a3=7fff030489c0 items=2 ppid=8699 pid=8773 auid=4294967295 uid=33 gid=33 euid=33 suid=33 fsuid=33 egid=33 sgid=33 fsgid=33 tty=(none) ses=4294967295 comm="sh" exe="/bin/dash" key=(null)	imap (8699) —execve→ sh (8773)
type=SYSCALL msg=audit(1583350370.218:45582193): arch=c000003e syscall=57 success=yes exit=3 a0=7fb186400988 a1=80000 a2=1 a3=7fb186608518 items=1 ppid=8773 pid=8774 auid=4294967295 uid=33 gid=33 euid=33 suid=33 fsuid=33 egid=33 sgid=33 fsgid=33 tty=(none) ses=4294967295 comm="netcat" exe="/bin/nc.traditional" key=(null)	sh (8773) —fork→ netcat (8774)

Fig. 2. The description of parsing system call behaviors of processes.

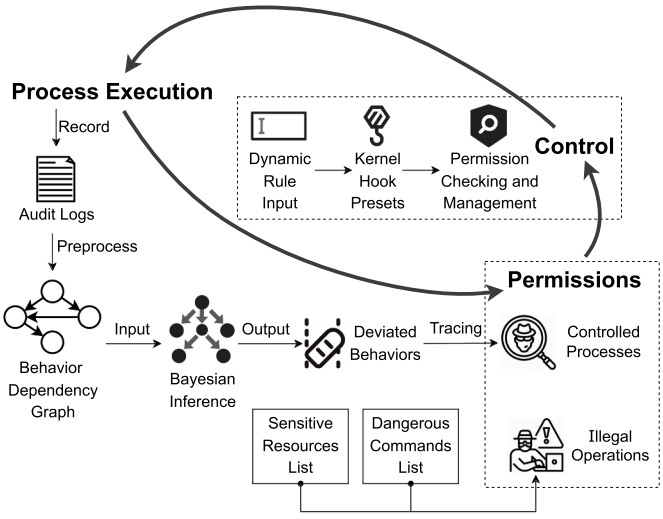


Fig. 3. The overview architecture of vaccine.

Description of Behaviors. System audit logs record comprehensive details regarding the interaction of the processes with resources in the operating system, including reading and writing operations, permission modifications, process forking and cloning, and network connections. Fig. 2 illustrates the audit logs of *SYSCALL* type along with the detailed parsing. Each log contains several fields, and we extract the values of syscall, ppid, and pid by regular expression matching to parse audit logs into system call behaviors. Additionally, we extract command and executable file paths to provide behavioral details. Similar to the anomaly detection methods, each behavior can be denoted as a triad $b =< s_c, o_c, sys_c >$, where s_c indicates the subject of a system call, represented by the parent process identified by ppid; o_c indicates the object, which is the subprocess or a file, with the information of name and path; sys_c indicates the operation of system call. The s_c performs sys_c operations on o_c, such as reading, opening, and writing a file, as well as forking and executing a process. Behaviors generated by legitimate program execution unaffected by malicious payloads conforms to benign patterns. Deviated patterns, of low similarity to benign patterns, result from exploitation or abnormal execution of programs.

Description of Permissions. The permissions of a process determine what resources it can access and what operations it is authorized to execute. We define the access control rule as a triad $r =< s_r, o_r, op_r >$, where s_r represents the subject, o_r represents the object, and op_r represents the operations. The mechanism of the permission bit, which is modified according to rules, prevents processes from abusing the read (r), write (w), and execute (x) system calls.

Goal. There is a correspondence between behaviors and permissions. Behaviors can only be executed if the required permissions are granted by the system. The core of vulnerability mitigation is to enforce process control to restrict the attack behaviors that result from exploiting vulnerabilities. Therefore, we dynamically adjust process permissions to deny malicious subprocess creation and block injected code execution. In this way, we prevent attacks from propagating, hence mitigating injection vulnerabilities without disrupting legitimate processes.

3.2.4. Threat model

Our threat model is similar to existing threat analysis work (Hassan et al., 2019; Milajerdi et al., 2019a; Alsaheel et al., 2021) and vulnerability mitigation (Milburn et al., 2022; Austin et al., 2018). We focus on injection vulnerabilities, not all forms of attacks, such as privilege escalations, which rely on deeper system compromises. Our design aims to mitigate injection vulnerabilities that are detected in a system but cannot be fixed. Vaccine aims at preventing attacks at the earliest possible point in the attack chain by dynamically enforcing fine-grained permission control at the process level, thereby limiting the initial impact scope of the injection vulnerability and effectively mitigating multiple attacks that may stem from it, making it resilient to evolving attacks. In our threat model, we assume that the audit logs have not been tampered with and that the vulnerabilities can be effectively detected. Both the audit framework and vulnerability information are trusted. Besides, the integrity of the enforcement mechanism is also significant to the effectiveness of Vaccine. As described in Section 2.2, we believe that the attacker initially gains access through application-level injection vulnerabilities, which do not grant elevated privileges to hijack eBPF program, while being protected by eBPF verfier.

4. Method design

To effectively mitigate injection vulnerabilities, we develop Vaccine, a method for dynamic and fine-grained process control. Vaccine determines process permissions based on behavioral deviation inference and enforces fine-grained process behavior control. The overview is illustrated in Fig. 3.

4.1. Analysis of control targets with behavioral deviations

The behavioral deviations of the process stemmed from a vulnerable program are a strong indicator that it has been exploited, requiring control to prevent potential attacks. Vaccine analyzes process behavior dependency graphs by GMM clustering and infers deviated behaviors using Bayesian inference to dynamically determine control targets.

4.1.1. Process behavior extraction

To better depict the behaviors of system calls between processes, Vaccine extracts and models process behaviors based on system call dependencies in audit logs. Vaccine distinguishes between normal operational patterns and anomalous behaviors with deviations that require control. Vaccine parses audit logs as dependency graphs and optimizes the graphs through pruning and partitioning. The flow is illustrated in Fig. 4.

Graph Construction. We consider processes and files as entities, identified by PID and file paths, respectively. Each system call log can be parsed into a behavior, represented as a triad $< s_c, o_c, sys_c >$. We take entities as nodes and system calls as edges to construct a dependency graph $G = (V, E)$, where V and E represent the sets of nodes and edges, respectively. The directions of edges are determined by the syscall. In the graph G, each edge, along with its associated parent and child nodes, represents a process behavior. The system dependency graph G illus-

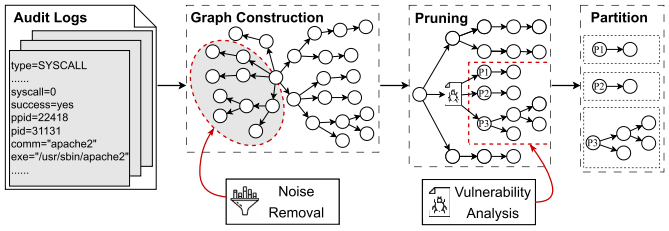


Fig. 4. The framework of audit logs preprocessing.

trates the interactions and dependencies between entities, which is the source for analyzing illegal permissions.

Graph Pruning and Partition. To reduce analysis overhead and focus on relevant behaviors, Vaccine ignores unnecessary dependencies. It selectively extracts behaviors occurring during the period of vulnerability exploitation while filtering out routine activities at the system level and global events unrelated to vulnerabilities, such as startup and shutdown procedures, antivirus scans, and kernel scheduled tasks. It enables Vaccine to focus on the behavioral deviations within processes. To further refine the scope, Vaccine filters processes associated with the vulnerable program, since the injected payload affects subprocesses of vulnerable programs. Using information from vulnerability disclosures, Vaccine determines the location of the vulnerable program. If explicit locations are unavailable, we infer them from class or function calls or details of the vulnerability mechanism in vulnerability descriptions.

After pruning, the dependency graph is segmented. According to the characteristics of injection vulnerability exploitation, we take the initial process of the vulnerable program as the root node and include all of its subprocesses recursively. Each partition corresponds to an independent process tree to ensure the integrity of the attack chain. Afterwards, we extract the process nodes from the graph and collect the edges from each node. Behavioral sequences consist of syscalls sorted by their timestamps and their associated objects for further analysis.

4.1.2. Process behavior deviation inference

Benign processes generated by the same program typically exhibit similar behavioral patterns, while processes injected with malicious payloads indicate deviation from normal execution for performing attacks, which necessitate permission control. Vaccine employs a clustering model to infer behavioral deviations dynamically through uncertainty estimation using Bayesian inference. Instead of precise detection of malicious behaviors, Vaccine uses process behavior embeddings and probabilistic modeling to determine whether a process exhibits deviation and requires control.

Graph Embedding. To obtain the vectorized representation of process behaviors for inference, Vaccine converts the behavior dependency graphs into matrices using Node2Vec. Node2Vec captures the relationships among syscall sequences, enabling similar behavioral patterns represented by similar vectors. Each node is embedded into a vector space, forming an embedding matrix. Vaccine further refines the representation using an Autoencoder to extract key behavioral features.

Bayesian Inference of Behavioral Deviation. Vaccine adopts GMM with Bayesian inference to quantify the behavioral deviation degree of process execution dynamically. The behavioral feature encoding of benign processes, derived from historical data, serves as the clustering centers. Our goal is to model the standard of benign behaviors and identified the deviated points as potential malicious activities. GMM exhibits sensitivity to outliers and possesses the capability to model the multi-modal nature of benign process behaviors, as a program can demonstrate several distinct patterns.

The deviation degree of a given process is measured by the cosine similarity between its embedding matrix and the benign cluster center. This approach is also used in other behavioral clustering analysis for calculation focusing on the similarity in the direction of behavioral patterns in high-dimensional embedding space rather than their magnitude. Specifically, given two matrices $A, B \in M$, Vaccine calculates their cosine similarity $S(A, B)$ as follows:

$$S(A,B) = \frac{A \cdot B}{\|A\| \times \|B\|} = \frac{\sum_{i,j,k}^{n} A_{ik} \cdot B_{kj}}{\sqrt{\sum_{k,i}^{n} (A_{ik})^2} \times \sqrt{\sum_{k,j}^{n} (B_{kj})^2}}. \tag{1}$$

Processes exhibiting high similarity in behaviors are clustered closely. Therefore, those near the benign clustering center indicate consistency with normal behavioral patterns. In contrast, processes that significantly deviate from the benign cluster are marked as requiring control. We calculate the prior probability $p(\theta)$ of a process exhibiting deviation, which is

$$p(\theta) = \frac{\sum_{benign}^{b} min(max(e^{-\lambda \cdot S(m_\theta, m_b)}, 0), 1)}{num(benign)}, \tag{2}$$

where θ is the process under observations and m_b is the embedding matrix generated by the behavioral graph of each benign process. To enhance sensitivity to deviations, we set the constant $\lambda = 1.5$, which is slightly large to amplify the change in prior probability $p(\theta)$ with the similarity $S(m_\theta, m_b)$.

Considering that injection attacks often involve modifying or hijacking benign processes, Vaccine continuously monitors process behaviors. As audit logs record process behaviors, Bayesian inference is applied to GMM. It dynamically integrates prior knowledge carried by historical benign behaviors, with real-time evidence, presented by current process behaviors, to output a posterior probability of deviation. Different from the traditional and complex Bayesian networks, we adopt Bayesian inference to perform a lightweight computation involving the product of probability derived from GMM and the prior distribution. Specifically, the probability is calculated as

$$p(\theta|X) = \frac{p(X|\theta)p(\theta)}{p(X)}, \tag{3}$$

where $p(\theta)$ is the prior distribution based on the historical behavioral data. $p(X|\theta)$ is the likelihood, indicating the probability of observing a given behavioral sequence under the assumption of deviating from normal behaviors. The posterior probability $p(\theta|X)$ indicates the probability of behavioral deviation according to the dynamic monitoring logs. Therefore, the posterior probability quantifies the necessity of enforcing control on the process. This dynamic inference mechanism refines the process control mechanism, ensuring that processes with behavioral deviations are controlled. Once the probability of behavioral deviation exceeds a predefined threshold, Vaccine enforces fine-grained control.

4.2. Fine-grained process behavior control

By integrating behavior deviation inference with permission control, Vaccine provides a dynamic process-level control mechanism, effectively mitigating injection vulnerabilities.

4.2.1. Policy analysis

Once a process exhibits behaviors that deviate from the normal patterns, indicating potential exploitation, it is imperative to restrict its permissions to contain the attack and prevent further propagation. Unlike post-incident recovery measures that attempt to repair damage after an attack, Vaccine controls process permissions proactively to prevent further exploitation, significantly reducing the attack impact. As shown in Fig. 5, we restrict processes from continually performing deviated operations and prevent potential attacks. The control mechanism is based on policy, which is dynamically determined based on the deviated and normal behaviors, ensuring that essential functionalities remain unaffected while blocking potential exploitation behaviors.

Our analysis reveals that the deviated process behaviors have a noticeable increase in the egress degree of nodes within the dependency graph. It represents the abnormally elevated quantity of child nodes. Specifically, these deviated behaviors include the following types: *1)*

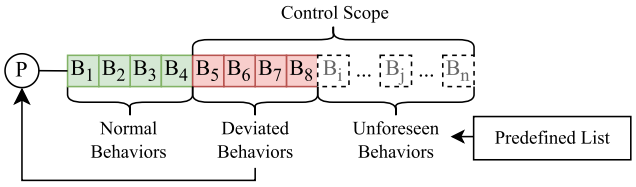


Fig. 5. Description of the goal of pre-defined policy.

Repeated unauthorized access attempts, such as permission probing and brute force attack. *2)* Access to objects typically not accessed by benign processes, which means that the accessed resources do not match the legitimate functionality. *3)* A significant increase in the frequency of the normal behaviors, such as unusually high frequency file access or network communication requests.

In operating systems, the inheritance of permissions by subprocesses depends on the UID/GID and capabilities of their parent processes. However, if a highly privileged process is compromised, its elevated privileges should not be indiscriminately propagated to its subprocesses. To mitigate this risk, policies define the set of restricted permissions and control specific process behaviors accordingly. These permissions are determined by their deviation from benign behaviors and a predefined list of sensitive resources and high-risk commands. For instance, if a process attempts to illegally read */etc/passwd* to escalate its privileges, access to files within the sensitive resource list is restricted. Similarly, if a process exhibits abnormal patterns of network communication, restrictions are enforced on the execution of related commands, such as *ifconfig* and *curl*. By restricting these critical high-risk operations while allowing other legitimate functionalities shared by benign processes, the control mechanism ensures that security is maintained without unnecessarily impacting system functionality.

4.2.2. Process control based on eBPF

Vaccine leverages eBPF to enforce fine-grained process permission control. Vaccine is designed to prevent attacks proactively and narrow the attack surface, rather than to respond to incidents after they occur. Therefore, it enforces a control mechanism by limiting the process creation from the parent process, instead of directly restricting the process behaviors.

The creation of a process can be divided into two phases, as shown in Fig. 6. Initially, the parent process initiates a *fork()* system call, which instructs the kernel to duplicate the current process and allocate a new PID. The kernel returns the PID of the subprocess to the parent process and a value of 0 to the subprocess. Subsequently, the subprocess initiates a *exec()* system call, prompting the kernel to load the specified program into the memory space of the subprocess, replacing the original program. The LSM hook checks this system call request and returns a value to the process. If the request is allowed, the return value is 0. If it is denied, the return value is -1. Our observation reveals that when rejected, the process addresses this error condition rather than proceeding with execution. Thus, the creation of subprocesses is blocked.

Based on the mechanism of process creation, we design the Algorithm 1 for security checks and process control via eBPF. We develop the function *se*, which operates in user space for policy management, and the function *sebpf*, which works in kernel space to enforce policies directly at the system call level. These functions can execute in parallel and dynamically load and enforce security policies, enabling process control without altering the kernel source code. We integrate the algorithm into the Linux kernel via eBPF through the *lsm_inode_permission* hook, which performs kernel permission verifications to control the reading, writing, and executing permissions of the processes. The algorithm dynamically enforces security policies by associating PID with fine-grained permission rules, effectively limiting unauthorized access.

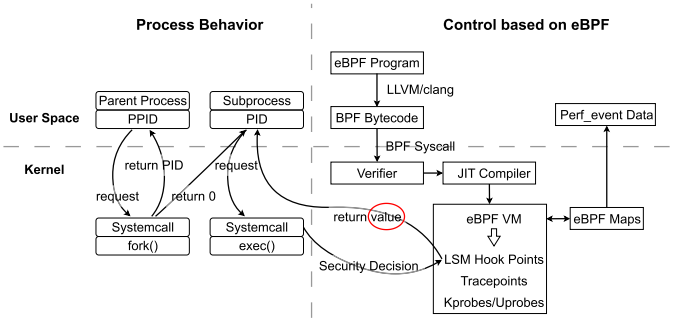


Fig. 6. Working mechanism of eBPF for process control.

Algorithm 1: Control algorithm using eBPF.

Input: pid, file_path, permissions
Output: program execution status

```
Function se(input):
    Initialize eBPF program;
    // Update policies in maps
    foreach input do
        PID ← input.pid;
        inode ← GetInode(input.file_path);
        permission ← input.permissions;
        key =< PID, inode>;
        maps.update(key,permission);
    Request to Load and attach eBPF to LSM hook;
// Triggered by LSM hook
Function sebpf(curr):
    PID ← curr.ppid;
    inode ← GetInode(curr.file_path);
    operation ← curr.permissions;
    PermissionValue = maps[< PID, inode>];
    if operation & *PermissionValue! = 0 then
        return -1; // Deny access
    else
        return ret; // Allow access
```

For example, a shell of PID 8774 exhibits deviated behaviors, and Vaccine aims at preventing its subprocesses from reading sensitive files, such as the password file. Following the specifications of *maps* in eBPF and the definition of permission, Algorithm 1 uses the triad *<8774, /etc/passwd, r>* as input, subsequently converted into a key-value pair. The key consists of *8774* as the low 32 bits and the inode of the */etc/passwd* file as the up 32 bits. The value represents a restriction on a specific system call. Our input *r* correlates with the permission flag of *MAY_READ*. This rule is added to the *maps* collection as a component of policy enforcement. When a process attempts to access the password file, the LSM hook intercepts the system call, verifies the PID of the associated process, and enforces predefined security policies. If the requested operation is restricted, the system call is denied, effectively blocking unauthorized access. The *PermissionValue* from *maps* is queried using these parameters, such as reading a file which also includes the *MAY_READ* flag. The algorithm verifies if the requested operation corresponds to the queried *PermissionValue* restricted by the policy rules. If they match up, the access request is denied, and the malicious behavior is prevented. Similarly, to prevent tampering with critical files, we restrict write access using the *MAY_WRITE* flag. Besides, Vaccine limits the unauthorized command execution with the *MAY_EXECUTE* flag, such as inputting $< 8774, /usr/bin/curl, x >$ restricts the permissions for the process to send and receive packets.

5. Experiment

In this section, we focus on evaluating the effectiveness and efficiency of Vaccine for injection vulnerability mitigation. In particular, we evaluate Vaccine by answering the following research questions (RQs):

RQ1: Is Vaccine accurate in controlling based on behavioral deviation? (§5.2)

RQ2: Is Vaccine effective at mitigating injection vulnerabilities? (§5.3)

RQ3: How much does Vaccine narrow the impact on benign behaviors? (§5.4)

RQ4: What is the latency loss and the overhead of Vaccine? (§5.5)

RQ5: Are the proposed components effective in enhancing the performance of Vaccine? (§5.6)

5.1. Experiment setup

5.1.1. Environment infrastructure

Vaccine is designed to dynamically control process execution and permissions through eBPF to mitigate injection attacks. The core of Vaccine is built on the *libbpf* framework (Libbpf, 2024), which provides an interface for eBPF-based process control. To analyze process behaviors and infer deviated processes, we utilize Python 3.7, employing the Node2vec library (0.4.3) (Node2Vec, 2024) for graph embedding and the sklearn library (1.4.2) (SKlearn, 2024) for clustering. These analysis components assist in guiding the enforcement of process control. Vaccine is deployed on a CentOS Stream 9 system with Linux kernel 5.14.0–480.el9.x86_64, with libelf (Libelf, 2024) and zlib (Zlib, 2024). Vaccine contains around 4K lines of code for process control and its supporting components.

5.1.2. Data collection

To evaluate the effectiveness of Vaccine, we universally select 40 injection vulnerabilities for Linux from the publicly available database CVE (CVE, 2024a). Ensuring diversity and representativeness, these vulnerabilities, detailed in Table 1, are selected solely based on the following criteria. First, they fall into standard injection vulnerability categories, including RCE, overflow, and command injection. Second, they have publicly available PoCs or malware exploits for reproducibility. Third, they represent a broad range of affected systems and applications, including libraries, commands, software, and services. Besides, we further statistic the severity scores and the corresponding patch release latency. On average, patches take several weeks to be available, with some high-impact vulnerabilities remaining unpatched indefinitely. Vaccine plays a crucial role by dynamically controlling processes during this time lag, effectively mitigating injection vulnerabilities before official patches are deployed.

To analyze attack behaviors, we collect audit logs that capture system events related to the exploitation of injection vulnerabilities and the execution of various attacks. This dataset includes PoCs exploitation as well as attack executions by 436 malware in 16 families (VirusTotal, 2024; VirusShare, 2024) which are predicated on the injection vulnerability exploits in Linux. Some families contain multiple malware samples, from which we randomly select thirty samples per family to prevent the duplicated collection of similar attacks from the same family. Since program execution times vary, we set a maximum execution time window of 15 min for the time interval per sampling cycle between triggering the vulnerability and sampling. It is for maintaining a manageable data size and ensuring comprehensive process behavior coverage. The filtered audit logs exceed 150GB and produce 5392 process behavioral dependency graphs, which serve as input. Besides, we obtain benign behavioral patterns through diverse legitimate inputs that do not trigger the injection vulnerabilities. We focus on the vulnerable program and monitor its status. Baseline updates are triggered adaptively upon monitoring a change in the version of the program, as this is an indicator of potential behavioral shifts. Afterwards, we retrain the model with the updated baseline.

Table 1
Dataset of injection vulnerabilities.

CVE ID	Types	Scope	Score	Patch Latency
CVE-2021-44228	RCE	Log4j	10.0 Critical	4h
CVE-2021-39926	Overflow	Wireshark	7.5 High	56h
CVE-2023-42117	RCE	exim mail	8.1 High	–
CVE-2016-10033	RCE	PHP Mailer	9.8 Critical	–
CVE-2017-5638	Overflow	Apache	9.8 Critical	163h
CVE-2014-6271	Injection	Apache	9.8 Critical	85h
CVE-2019-5418	Leakage	File System	7.5 High	1165h
CVE-2019-9858	RCE	Horde mail	8.8 High	1631h
CVE-2019-8942	RCE	WordPress	8.8 High	247h
CVE-2023-3745	RCE	ImageMagick	5.5 Medium	201h
CVE-2014-9093	Overflow	libreoffice	7.5 High	∞
CVE-2023-26435	Overflow	libreoffice	5.0 Medium	1512h
CVE-2022-32742	RCE	Samba Server	4.3 Medium	–
CVE-2024-52010	Injection	web ssh	8.6 High	∞
CVE-2019-10803	Injection	git	9.8 Critical	∞
CVE-2012-2807	Overflow	libxml2	6.8 Medium	1248h
CVE-2017-17541	Overflow	libjpeg	6.1 Medium	–
CVE-2015-0235	Overflow	glibc	10 Critical	1195h
CVE-2009-4881	Overflow	glibc	5.0 Medium	3967h
CVE-2021-4034	LPE	pkexec	7.8 High	1280h
CVE-2018-1002206	Traversal	Filesystem	5.5 Medium	–
CVE-2024-48990	LPE	needrestart	7.8 High	123h
CVE-2024-48991	LPE	needrestart	7.8 High	123h
CVE-2024-48992	LPE	needrestart	7.8 High	123h
CVE-2024-10224	Injection	needrestart	5.3 Medium	123h
CVE-2024-11003	RCE	needrestart	7.8 High	123h
CVE-2022-25765	Injection	pdfkit	7.3 High	12744h
CVE-2020-8250	LPE	pulsesvc	7.8 High	2568h
CVE-2021-3156	Overflow	sudo	7.8 High	181h
CVE-2023-4490	RCE	WP plugin	9.8 Critical	–
CVE-2022-0185	Overflow	Filesystem	8.4 High	8h
CVE-2020-14386	LPE	socket	7.8 High	1948h
CVE-2023-1389	Injection	web interface	8.8 High	∞
CVE-2017-9430	Overflow	dnstracer	9.8 Critical	∞
CVE-2017-13089	Overflow	wget	8.8 High	73h
CVE-2023-1829	UAF	tcindex	7.8 High	2615h
CVE-2019-11815	RCE	net namespace	8.1 High	126h
CVE-2019-2941	LPE	Hyperion	4.0 Medium	2342h
CVE-2022-47941	Leakage	ksmbd	7.5 High	38h
CVE-2022-47942	Overflow	ksmbd	8.8 High	37h

5.1.3. Baselines

Vaccine enforces dynamic fine-grained and process controls to mitigate injection vulnerabilities. We compare Vaccine with existing analysis and control techniques, focusing on two key aspects: (1) Vaccine reduces the disruptions to legitimate process behaviors through dynamic fine-grained control; (2) Vaccine analyzes behavioral deviation effectively with lower latency loss. For the first aspect, we employ Tetragon (Cilium, 2024), an advanced policy enforcement engine based on eBPF developed in 2022. We also modify the policy of SELinux (Mccarty, 2005), which is an implementation of mandatory access control in Linux systems that minimizes the resources that can be accessed through policy enforcement, to mitigate injection vulnerabilities. The comparison illustrates that Vaccine achieves better control (Shen et al., 2023) with fewer restricted behaviors. For the second aspect, we employ three methods of attack behavioral analysis using audit logs and graphical analysis, which are NoDoze (Hassan et al., 2019), Unicorn (Han et al., 2020), and DEPCOMM (Xu et al., 2022). We use them to prove that Vaccine has less latency loss.

5.2. RQ1: Accuracy of behavioral deviation analysis for permission determination

We evaluate behavioral deviation analysis, which serves as the foundation for dynamic process permission determination. In each scenario

Table 2
Performance of behavioral deviation analysis.

Scenario	#Ben.	#Dev.	#Contr.	Precision	Recall	F1
s-20	20	4	3	0.99	1.0	0.995
s-50	50	4	4	1.0	1.0	1.0
s-100	100	4	4	1.0	1.0	1.0
s-200	200	4	4	1.0	1.0	1.0
s-400	400	4	4	1.0	1.0	1.0
s-800	800	4	4	1.0	1.0	1.0
Average	–	–	–	0.998	1.0	0.999

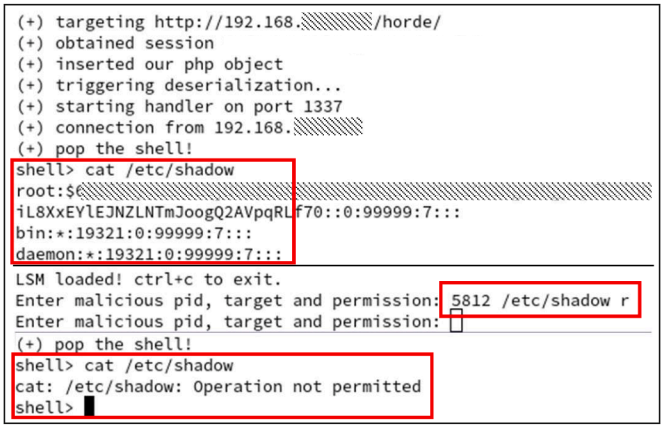


Fig. 7. The effect of denying malicious behaviors.

of exploiting injection vulnerabilities, we monitor processes when executing PoCs and malware to produce diverse attack vectors. Meanwhile, we execute legitimate inputs to obtain benign behavioral patterns. Vaccine analyzes process behaviors and infers the deviations, which inform subsequent permission adjustments. We take those that exhibit deviations as controlled processes, while those that do not exhibit deviations are benign processes. To assess the performance of behavioral deviation analysis, we evaluate it across datasets of varying scales and calculate the average accuracy. The results are presented in Table 2.

Column 1 in Table 2 shows the number of behaviors in test scenarios, and Columns 2, 3, and 4 denote the quantities of benign processes, processes exhibiting behavioral deviations, and processes requiring control, analyzed by Vaccine, respectively. Table 2 further demonstrates the precision, recall, and F1-score. The results indicate that Vaccine effectively distinguishes processes based on behavioral deviations, ensuring precise permission adjustments. With sufficient benign behavioral patterns, Vaccine achieves accurate analysis without false positives and supports fine-grained control, apart from cases where the limited data compromises model accuracy.

5.3. RQ2: Effectiveness of mitigating injection vulnerabilities

5.3.1. Illustration of effects of control

We demonstrate the effect of preventing attacks from executing using Vaccine, as depicted in Fig. 7. After analyzing behavioral deviations and dynamically enforcing the rules, the injected reverse shell changes from reading sensitive files to displaying *Operation not permitted*. By dynamically controlling access to potentially harmful operations, Vaccine effectively narrows the attack surface and denies attack behaviors to mitigate injection vulnerabilities. When each injection attack of the injection vulnerability is denied to execute due to a lack of permissions, we consider the vulnerability to be fully mitigated.

5.3.2. Case study

We illustrate the effort of Vaccine on injection vulnerability mitigation through process control with two specific scenarios.

Mitigation of DDoS. DDoS attacks disrupt normal responses and services by depleting substantial resources, which are typically high-intensity and short-duration. Vaccine analyzes the deviations from normal behaviors and dynamically controls excessive network interactions. Once an attack process exhibits deviations, such as frequent packet transmissions or repeated socket connections, Vaccine modifies the network access permissions of the process. Controlling permissions for sending packets or terminating its network connection can prevent attacks. Compared to conventional interception methods relying on IP-based or traffic patterns analysis, Vaccine's control mechanism is more precise and responds rapidly, reducing the impact on legitimate communications.

Mitigation of APT. In APT threats, the attackers remain in the target system for an extended duration and execute multi-stage attacks, as described in Section 3.1. Vaccine enforces permission control on processes attempting unauthorized actions that deviate from normal behaviors, such as initiating a reverse connection or modifying critical system resources. Subsequent malicious accesses are prevented by dynamically restricting their permissions using eBPF. The dynamic control demonstrates Vaccine's positive efforts in narrowing the attack surface. Compared with the traditional malware detection methods based on signatures, controlling process permissions based on system call sequences offers a more effective defense mechanism against evolving threats.

Mitigation of Composite Attacks. A single attack step typically cannot directly achieve the attack objective. Therefore, attackers opt for composite attacks, such as exploiting an injection vulnerability to obtain a reverse shell for further execution of system scanning, privilege escalation, lateral movement, and data theft. During the phase of system scanning, the process executes commands, such as *ipconfig*, for network configuration probing that significantly deviated from the normal behavioral profile of a web server. Vaccine analyzes the behavioral deviations and triggers fine-grained control enforcement through eBPF. Specifically, Vaccine denies operations for the entire process tree to sensitive files and blocks connections to external IP. Therefore, privilege escalation is disabled due to failure to read the *passwd* file, and the data theft could not execute as the network connection was denied. Through early behavioral deviation analysis and dynamic process control, Vaccine mitigate multi-stage attack chains by governing the entire affected process tree.

5.4. RQ3: Impact on legitimate process behaviors

In this section, we illustrate the effectiveness of Vaccine by reducing the impact on legitimate behaviors through dynamic fine-grained process behavior control. The control policy of Vaccine is generated in a semi-predefined way, which is a combination of dynamic analysis of behavioral deviations and predefined lists. The dynamic analysis of behavioral deviations ensures that legitimate behaviors are not interfered with, and the comprehensive predefined list ensures that processes injected with attack payloads do not perform other attacks. In most cases, attack patterns of the injection vulnerabilities are hard to predict and without obvious intent. Therefore, we enforce a strict control policy that guarantees the mitigation of injection vulnerabilities to proactively control processes. To prevent occurred attack propagation, Vaccine ensures that all processes invoked by the compromised process are also subjected to permission controls, while processes that have not been invoked remain unaffected. However, even in the most restrictive enforcement situation, Vaccine guarantees the uninterrupted operation of unrelated programs, and the majority of benign processes initiated by the program are not affected, thereby reducing the unintended disruptions to legitimate behaviors.

To further illustrate the advantages of Vaccine, we compare it with SELinux and Tetragon. SELinux is a native fine-grained access control tool for Linux. SELinux enforces access control policy by assigning security labels to files depending on their paths. Generally, processes initiated by the program inherit its security labels and possess identical

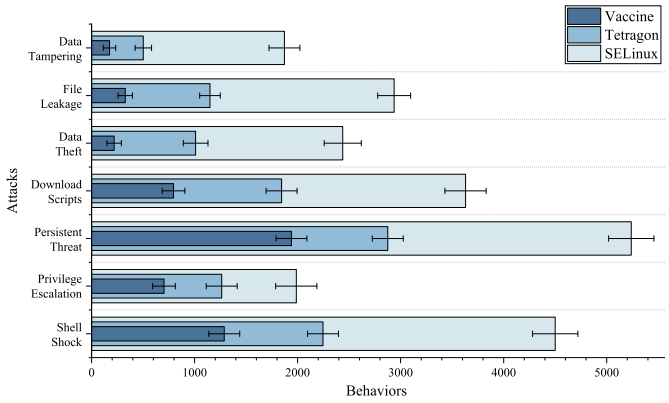


Fig. 8. Various attacks and restricted behaviors comparison.

permissions. Consequently, while SELinux provides isolation, its control is indiscriminate, because it impacts all child processes of the vulnerable program, regardless of whether they are benign or malicious. It leads to a higher number of restricted legitimate behaviors to ensure security. In contrast, the process control of Vaccine demonstrates that it dynamically distinguishes between benign and malicious processes within the same program context, which enables enforcement with significantly fewer disruptions to legitimate operations. Furthermore, Vaccine simplifies the manual customization of complex policies, compared to SELinux, which requires compiling configurations to load and enforce policies.

Tetragon mounts probes on system call paths for fine-grained observation and control based on eBPF. Different from Vaccine, Tetragon relies on predefined static policy matching to control each system call. For example, once a process is detected attempting to connect network by executing */bin/nc*, Tetragon sends *SIGKILL* to terminate the process. It leads to indiscriminate interruption of all processes executing specific system calls and unnecessary restrictions on those benign processes. In comparison, Vaccine analyzes behavioral sequences in a high dimension, instead of a single system call. Dynamic determinations of process permissions based on behavioral deviations enable control policies to incorporate contextual features, thereby reducing disruptions to legitimate process execution.

We compare the amount of restricted behaviors between Vaccine and Tetragon, as demonstrated in Fig. 8. The light-colored, normal-colored, and dark-colored bars represent the average amount of denial behaviors when performing control using SELinux, Tetragon, and Vaccine, respectively. Both of them deal with identical attacks and enforce control, ensuring mitigation of injection vulnerabilities. The results indicate that Vaccine significantly reduces restricted legitimate behaviors compared to SELinux and Tetragon, where SELinux limits the largest number of behaviors. For example, in the scenario of data tampering, 502 behaviors are restricted using the static policy of Tetragon and 1879 behaviors are restricted using program control of SELinux, while only 174 behaviors are restricted with Vaccine. The reduction rate is 65.3 %, indicating that Vaccine ensures more than half of restricted behaviors by Tetragon execute normally, leading to less impact on legitimate behaviors. Across various attack scenarios, Vaccine reduces the restricted legitimate behaviors by 32 % to 78 %, demonstrating its ability to selectively enforce security policies while preserving system functionality.

5.5. RQ4: System performance

5.5.1. System overhead

In order to demonstrate the feasibility of Vaccine in a practical system, we analyze its overhead caused by analysis and control based on eBPF. We employ Vaccine in a dual-core Intel Core i7-12700 system equipped with 16GB RAM. Vaccine reduces resource consumption by filtering and analyzing behaviors associated with vulnerabilities, greatly reducing overhead. The results indicate that the average CPU usage is around 9.4 % and memory consumption remains below 15 % for the analysis of behavioral deviations. Even under heavy workload conditions, where the number of behaviors in the training set reaches around 200, CPU usage remained below 13 %. The increase in CPU usage is attributed to extensive file access and system calls, which require to be processed. Besides, unlike traditional access control mechanisms that require frequent kernel-user space transitions, Vaccine is based on lightweight in-kernel eBPF. The duration of executing LSM hooks is short, and the overhead is for permissions verification, conducted solely when triggered. The overhead incurred by the eBPF control has been evaluated to be no more than 1 % of CPU usage and around 12MB of memory consumption.

5.5.2. Latency loss

Table 1 illustrates the significant delay between vulnerability disclosure and the release of its patches. During this exposure window, attackers can exploit the vulnerability, putting systems at substantial risk. Unlike patches that provide a permanent fix but require considerable time to develop and deploy, Vaccine offers an immediate response by dynamically enforcing process control upon detecting behavioral deviations. It significantly reduces the attack surface before patches are available, providing a critical defense mechanism to mitigate injection vulnerabilities.

Process control based on eBPF incurs negligible latency, and most delays lie in behavioral deviation analysis. In real attack scenarios, the efficiency of behavioral deviation analysis directly correlates with the pace of attack execution. Therefore, we analyze the latency loss based on the volume of process behaviors. Vaccine continuously updates the probability of deviated behaviors as audit logs processing. We set a threshold for the probability, above which is judged to require control. A low threshold enhances efficiency but also increases the false positives. To control accurately, the threshold is set to at least 0.407 ($e^{-1.5*0.6}$). Under this configuration, triggering control necessitates at least 60 deviated system call behaviors. For most attacks, a fully executed attack typically generates at least 1300 system calls that deviate from the normal behavioral patterns. It proves that Vaccine enables timely and effective mitigation of injection vulnerabilities.

To further illustrate the latency, we count the time in retrospective evaluations using complete attack logs and compare three behavioral analysis methods with Vaccine. All of them construct dependency graphs based on audit logs and analyze deviated behaviors. Unicorn trains K-medoids clustering algorithm using sequences of benign behaviors and compares the incrementally captured behaviors with clustering centers to determine anomalies. NoDoze scores and ranks the deviations of the partitioned subgraphs based on a database of frequent events. DEPCOMM compresses and summarizes communities centered on processes and calculates the priority based on the critical path of known point-of-interest events and matches the dependencies between the associated communities with the kill chain. The primary goal of Vaccine is to achieve a dynamic control mechanism for injection vulnerability mitigation. To achieve this, Vaccine requires computationally lightweight based on pursuing accuracy. In comparison, Vaccine adopts GMM, which is a low time-complexity clustering model, and Bayesian inference, which incrementally analyzes behavioral deviations to save latency.

Fig. 9 quantifies the behavioral deviation analysis latency, which is calculated from attack initialization to deviated processes determined, across seven attack scenarios. The plot employs minute-scale radial coordinates, where proximity to the origin indicates lower latency. It reveals that Vaccine achieves a response latency of 4 min on average, whereas the comparative methods exhibit longer latency, because their reliance on deep feature extraction introduces inherent delays. It is worth noting that Vaccine focuses on proactively preventing rather than blocking attacks, requiring strict adherence to real-time permission determination

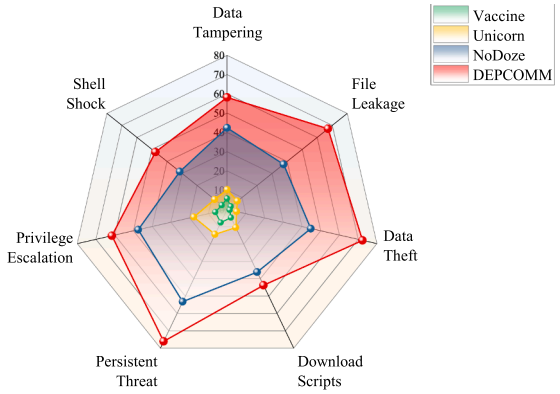


Fig. 9. Various attacks and their restriction behaviors.

Table 3
Performance of behavioral analysis under different thresholds of λ.

Threshold	Precision	Recall	F1-score	FNR	FPR
$\lambda = 0.2$	99.52 %	89.71 %	94.36 %	10.27 %	0.43 %
$\lambda = 0.5$	99.17 %	94.55 %	96.79 %	5.45 %	0.79 %
$\lambda = 1$	98.83 %	97.62 %	98.25 %	2.29 %	1.16 %
$\lambda = 1.5$	98.89 %	99.73 %	99.31 %	0.32 %	1.41 %
$\lambda = 2$	96.72 %	99.49 %	98.08 %	0.51 %	3.38 %
$\lambda = 4$	93.20 %	99.34 %	96.16 %	0.66 %	7.25 %

for effective process control and injection vulnerability mitigation. Compared to commonly used models for anomaly detection, such as VAEs, which typically require large amounts of data for meaningful representation learning and are more complex to implement and interpret, and the methods mentioned above, our approach is simpler, more efficient and aligns with demand for real-time control.

5.6. RQ5: Ablation study

We compare the performance of Vaccine under different threshold λ and illustrate the rationalization of analysis based on behavioral deviations through the clustering and similarity metrics. These results are generated in mitigating vulnerability CVE-2021-44228.

The threshold, which control the degree of amplifying the change in prior probability with the cosine similarity, is denoted as λ. The performance of at different thresholds is shown in Table 3. As λ increases, FPR gradually increases while the underreporting represented by FNR decreases, in addition to the rebound caused by the rapid surge in FPR. The selection of $\lambda = 1.5$ is empirically determined to achieve the optimal balance between high recall and precision, resulting in the highest F1-score. It ensures sensitivity to attacks without introducing excessive over-classification that influence the execution of legitimate functions.

Fig. 10 shows the similarity matrix for the behavioral dependency graphs of different processes in a system. We acquire the behaviors of benign processes and processes exploiting the vulnerability, including PoCs and malware. These processes stem from the same vulnerable program. We capture and analyze their comprehensive behavioral dependency graph post-execution. The matrix illustrates that the behaviors of benign processes exhibit similarities, while the processes generated by the same PoC also demonstrate similar behaviors. Besides, malware in the same family exhibits similar behaviors due to similar attack patterns. These processes form distinct behavioral clusters, indicating that behavioral deviations can serve as an effective basis for permission determination.

Fig. 11 further supports this observation by visualizing clustering results with marginal histograms. We project the feature matrix embedded in their behavioral dependency graph into a two-dimensional representation. Its horizontal and vertical coordinates represent feature dimensions after dimensionality reduction. The marginal histograms represent

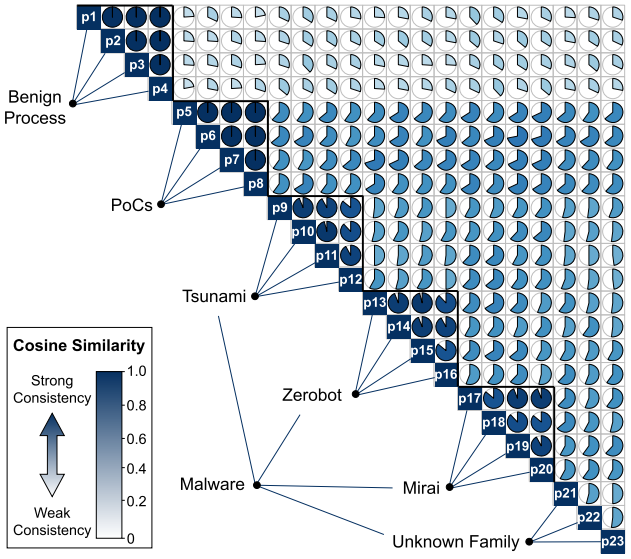


Fig. 10. The dependency graph similarity of processes.

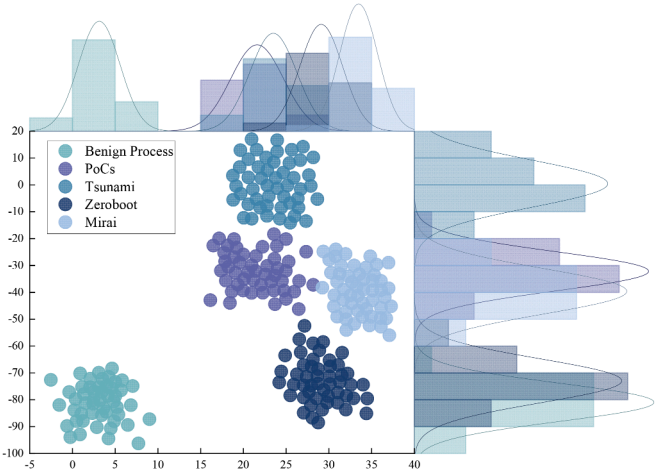


Fig. 11. The clustering of benign and malicious processes.

the distribution of samples of each category on this feature. The results indicate that process behaviors generated by the same exploitation of vulnerability are distinctly clustered, demonstrating consistency of execution. While different categories of malicious behaviors exhibit varying degrees of similarity in their feature projection distributions, all the malicious distributions deviate significantly from benign process distributions.

6. Discussion

Significance. This paper proposes a novel approach to injection vulnerability mitigation through dynamic process control, distinguishing it from conventional security mechanisms. Vaccine dynamically controls process permissions in response to behavioral deviations, effectively reducing the attack surface before official patches are available. It is worth noting that our innovation lies in fine-grained process control targeted at vulnerability mitigation using efficient and lightweight analysis, rather than in syscall anomaly detection. Experiments demonstrate that Vaccine successfully mitigates injection vulnerabilities while reducing disruptions to legitimate behaviors by enforcing dynamic and fine-grained control. This approach offers a security enhancement in the absence of patches.

Objectives and Targets. The primary goal of Vaccine is to achieve fine-grained control of deviated behaviors to mitigate attacks that exploit injection vulnerabilities. In this paper, Vaccine focuses on injection vulnerabilities, which permit unauthorized code execution by improperly handling external input. These attacks exploiting vulnerabilities alter the behaviors of existing processes, particularly their system call behaviors, which Vaccine can dynamically analyze and control. It is worth noting that Vaccine does not address vulnerabilities that do not affect process behavior, such as kernel-level attacks. To evaluate the applicability of Vaccine, we systematically analyze 40 publicly disclosed injection vulnerabilities, which are selected to cover diverse and representative real-world cases, across a broad range of types and affected components. While the experimental results demonstrate the strong effectiveness of Vaccine within the scope of injection vulnerabilities, we acknowledge that future work is required to extend coverage to other classes of vulnerabilities.

Illustration of False Positives. In host environments, benign processes spawned by the same program typically follow relatively consistent behavioral patterns, proved in Section 5.6. Vaccine further analyzes by dynamically updating deviation probabilities based on real-time audit logs, ensuring adaptability to context and reducing false determinations. However, the strict policy inevitably leads to false positives, where benign processes are determined as requiring control. Besides, there are certain special circumstances during execution, such as automatic program updates, which cause false positives. Blocking them is inappropriate for the normal execution and security of the system. However, even in this scenario, the control mechanism of Vaccine reduces disruption. Unlike Tetragon, which terminates processes, Vaccine enforces selective permission restrictions. For example, a benign process, which is misclassified and erroneously controlled by Vaccine, retains read permission for legitimate files, but is restricted from accessing sensitive resources. It ensures that legitimate functionalities remain unaffected while blocking attack vectors.

Limitations. Vaccine achieves dynamic process-level control to mitigate injection vulnerability. However, the following limitations should be considered. Vaccine determines policy by analyzing behavioral deviations compared to the benign processes of the same program. Therefore, it is essential to gather multiple benign process behaviors as prior information. Experiments demonstrate that a limited quantity of samples will influence the accuracy of behavioral deviation analysis. In the experiment, we repeatedly execute the program with several legitimate inputs and obtain benign process behaviors. However, it would be costly in a real environment. Utilizing data generation techniques to obtain additional behavioral information is also our future work. Besides, the baseline also includes feedback led by administrator where misclassified benign behaviors can be added to the baseline. This periodic examination, while incurring manual overhead, corrects anomalous legitimate behaviors, thereby extending the baseline in extreme system states. These updated behaviors are used to retrain GMM clustering model for accurate analysis.

7. Conclusion

It is essential to mitigate attacks exploiting vulnerabilities, especially in cases where it is difficult to obtain patches. To this end, we propose Vaccine, an injection vulnerability mitigation method based on the vulnerability-tolerant mechanism. The method leverages behavioral deviations to determine permissions and dynamically enforces process-level access control using eBPF. The experiment results indicate that Vaccine can effectively prevent injection attacks and mitigate injection vulnerabilities, while reducing the impact on benign behaviors. Compared to other methods, Vaccine provides a fine-grained approach, which reduces the impact on benign behaviors and incurs low overhead.

CRediT authorship contribution statement

Hanyu Wang: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Formal analysis; **Aimin Yu:** Writing – review & editing, Formal analysis; **Lifang Xiao:** Writing – review & editing, Data curation; **Lixin Zhao:** Writing – review & editing; **Xu Cao:** Writing – review & editing, Software, Validation; **Dan Meng:** Writing – review & editing.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research is supported by the Project on Dynamic Regulation of Complex Environmental Resouces, Institute of Information Engineering, Chinese Academy of Sciences, Grant No. E3Z0101205.

References

Alsaheel, A., Nan, Y., Ma, S., Yu, L., Walkup, G., Celik, Z.B., Zhang, X., Xu, D., 2021. ATLAS: a sequence-based learning approach for attack investigation. In: 30th USENIX Security Symposium (USENIX Security 21). USENIX Association, pp. 3005–3022. https://www.usenix.org/conference/usenixsecurity21/presentation/alsaheel.

Audit, 2024. Linux kernel audit subsystem. https://github.com/linux-audit/audit-kernel.

Austin, T., Bertacco, V., Kasikci, B., Malik, S., Tiwari, M., 2018. Vulnerability-tolerant secure architectures. In: Proceedings of the International Conference on Computer-Aided Design. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3240765.3273057

Cao, W., Xia, C., Peddinti, S.T., Lie, D., Taft, N., Austin, L.M., 2021. A large scale study of user behavior, expectations and engagement with android permissions. In: 30th USENIX Security Symposium (USENIX Security 21). USENIX Association, pp. 803–820. https://www.usenix.org/conference/usenixsecurity21/presentation/cao-weicheng.

Chen, H.-C., Sun, J.-Z., Wu, Z.-D., 2010. Dynamic forensics system with intrusion tolerance based on hierarchical coloured petri-nets. In: 2010 International Conference on Broadband, Wireless Computing, Communication and Applications, pp. 660–665. https://doi.org/10.1109/BWCCA.2010.151

Choraria, M., Chattopadhyay, A., Mitra, U., Ström, E.G., 2022. Design of false data injection attack on distributed process estimation. IEEE Trans. Inf. Forensics Secur. 17, 670–683. https://doi.org/10.1109/TIFS.2022.3146078

Cilium, 2024. Tetragon: eBPF-based security observability and runtime enforcement. https://tetragon.io/.

Connor, E., McDaniel, T., Smith, J.M., Schuchard, M., 2020. PKU pitfalls: attacks on PKU-based memory isolation systems. In: 29th USENIX Security Symposium (USENIX Security 20). USENIX Association, pp. 1409–1426. https://www.usenix.org/conference/usenixsecurity20/presentation/connor.

CVE, 2024a. CVE vulnerabilities. https://nvd.nist.gov/vuln.

CVE, 2024b. SecurityScorecard. CVE vulnerability statistics. https://www.cvedetails.com/_cf_chl_rt_tk=bgTuFb0Ov5O7Unp8E1B5w6nkoEGnkk43fvHZ5wc92_U-1726798744-0.0.1.1-4202.

Daffalla, A., Bohuk, M., Dell, N., Bellini, R., Ristenpart, T., 2023. Account security interfaces: important, unintuitive, and untrustworthy. In: 32nd USENIX Security Symposium (USENIX Security 23). USENIX Association, Anaheim, CA, pp. 3601–3618. https://www.usenix.org/conference/usenixsecurity23/presentation/daffalla.

Dejaeghere, J., Gbadamosi, B., Pulls, T., Rochet, F., 2023. Comparing security in eBPF and webassembly. In: Proceedings of the 1st Workshop on EBPF and Kernel Extensions. Association for Computing Machinery, New York, NY, USA, p. 35–41. https://doi.org/10.1145/3609021.3609306

DeMarinis, N., Williams-King, K., Jin, D., Fonseca, R., Kemerlis, V.P., 2020. Sysfilter: automated system call filtering for commodity software. In: 23rd International Symposium on Research in Attacks, Intrusions and Defenses (RAID 2020). USENIX Association, San Sebastian, pp. 459–474. https://www.usenix.org/conference/raid2020/presentation/demarinis.

Ding, H., Zhai, J., Nan, Y., Ma, S., 2023. AIRTAG: towards automated attack investigation by unsupervised learning with log texts. In: 32nd USENIX Security Symposium (USENIX Security 23). USENIX Association, Anaheim, CA, pp. 373–390. https://www.usenix.org/conference/usenixsecurity23/presentation/ding-hailun-airtag.

Feng, S., Wu, Y., Xue, W., Pan, S., Zou, D., Liu, Y., Jin, H., 2024. FIRE: combining multi-stage filtering with taint analysis for scalable recurring vulnerability detection. In: Balzarotti, D., Xu, W. (Eds.), 33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14–16, 2024. USENIX Association. https://www.usenix.org/conference/usenixsecurity24/presentation/feng-siyue.

Gaidis, A.J., Atlidakis, V., Kemerlis, V.P., 2023. SysXCHG: refining privilege with adaptive system call filters. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, p. 1964–1978. https://doi.org/10.1145/3576915.3623137

Ghavamnia, S., Palit, T., Mishra, S., Polychronakis, M., 2020. Temporal system call specialization for attack surface reduction. In: 29th USENIX Security Symposium (USENIX Security 20). USENIX Association, pp. 1749–1766. https://www.usenix.org/conference/usenixsecurity20/presentation/ghavamnia.

Han, X., Pasquier, T. F.J.M., Bates, A., Mickens, J., Seltzer, M.I., 2020. Unicorn: runtime provenance-based detector for advanced persistent threats. In: 27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23–26, 2020. The Internet Society. https://www.ndss-symposium.org/ndss-paper/unicorn-runtime-provenance-based-detector-for-advanced-persistent-threats/.

Hassan, W.U., Guo, S., Li, D., Chen, Z., Jee, K., Li, Z., Bates, A., 2019. Nodoze: combatting threat alert fatigue with automated provenance triage. In: 26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24–27, 2019. The Internet Society. https://www.ndss-symposium.org/ndss-paper/nodoze-combatting-threat-alert-fatigue-with-automated-provenance-triage/.

Hassan, W.U., Noureddine, M.A., Datta, P., Bates, A., 2020. OmegaLog: high-fidelity attack investigation via transparent multi-layer log analysis. In: 27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23–26, 2020. The Internet Society. https://www.ndss-symposium.org/ndss-paper/omegalog-high-fidelity-attack-investigation-via-transparent-multi-layer-log-analysis/.

Herber, C., Richter, A., Wild, T., Herkersdorf, A., 2014. A network virtualization approach for performance isolation in controller area network (CAN). In: 2014 IEEE 19th Real-Time and Embedded Technology and Applications Symposium (RTAS), pp. 215–224. https://doi.org/10.1109/RTAS.2014.6926004

Hough, K., Welearegai, G., Hammer, C., Bell, J., 2020. Revealing injection vulnerabilities by leveraging existing tests. In: Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering. Association for Computing Machinery, New York, NY, USA, p. 284–296. https://doi.org/10.1145/3377811.3380326

Humphries, J.T., Natu, N., Chaugule, A., Weisse, O., Rhoden, B., Don, J., Rizzo, L., Rombakh, O., Turner, P., Kozyrakis, C., 2021. GhOSt: fast & flexible user-space delegation of linux scheduling. In: Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles. Association for Computing Machinery, New York, NY, USA, p. 588–604. https://doi.org/10.1145/3477132.3483542

Jaeger, T., Sailer, R., Zhang, X., 2003. Analyzing integrity protection in the SELinux example policy. In: Proceedings of the 12th USENIX Security Symposium, Washington, D.C., USA, August 4–8, 2003. USENIX Association. https://www.usenix.org/conference/12th-usenix-security-symposium/analyzing-integrity-protection-selinux-example-policy.

Ji, Y., Lee, S., Downing, E., Wang, W., Fazzini, M., Kim, T., Orso, A., Lee, W., 2017. Rain: refinable attack investigation with on-demand inter-process information flow tracking. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, p. 377–390. https://doi.org/10.1145/3133956.3134045

Jim Solomon Raja, D., Hemavathi, N., Sriranjani, R., Arulmozhi, P., 2025. Integrated game theoretic and honeypot based distributed denial of service attack detection and mitigation in advanced metering infrastructure. IEEE Trans. Instrum. Meas. 74, 1–10. https://doi.org/10.1109/TIM.2025.3556910

Jin, D., Gaidis, A.J., Kemerlis, V.P., 2024. BeeBox: hardening BPF against transient execution attacks. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, Philadelphia, PA, pp. 613–630. https://www.usenix.org/conference/usenixsecurity24/presentation/jin-di.

Kermabon-Bobinnec, H., Jarraya, Y., Wang, L., Majumdar, S., Pourzandi, M., 2024. Phoenix: surviving unpatched vulnerabilities via accurate and efficient filtering of syscall sequences. In: 31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26- March 1, 2024. The Internet Society. https://www.ndss-symposium.org/ndss-paper/phoenix-surviving-unpatched-vulnerabilities-via-accurate-and-efficient-filtering-of-syscall-sequences/.

Lee, H., Song, C., Kang, B.B., 2023. Harnessing the x86 intermediate rings for intra-process isolation. IEEE Trans. Dependable Secur. Comput. 20 (4), 3251–3268. https://doi.org/10.1109/TDSC.2022.3192524

Lee, Y.-T., Enck, W., Chen, H., Vijayakumar, H., Li, N., Qian, Z., Wang, D., Petracca, G., Jaeger, T., 2021. PolyScope: multi-policy access control analysis to compute authorized attack operations in android systems. In: 30th USENIX Security Symposium (USENIX Security 21). USENIX Association, pp. 2579–2596. https://www.usenix.org/conference/usenixsecurity21/presentation/lee-yu-tsung.

Li, T., Liu, X., Qiao, W., Zhu, X., Shen, Y., Ma, J., 2024. T-trace: constructing the APTs provenance graphs through multiple syslogs correlation. IEEE Trans. Dependable Secur. Comput. 21 (3), 1179–1195. https://doi.org/10.1109/TDSC.2023.3273918

Li, Y., Huang, C., Yuan, L., Ding, Y., Cheng, H., 2020. Aspgen: an automatic security policy generating framework for apparmor. In: 2020 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom), pp. 392–400. https://doi.org/10.1109/ISPA-BDCloud-SocialCom-SustainCom51426.2020.00075

Libbpf, 2024. Libbpf. https://github.com/libbpf/libbpf.

Libelf, 2024. Libelf. https://www.ibm.com/docs/en/zos/2.1.0?topic=clu-libelf.

Liu, H., Reiter, M.K., Gong, N.Z., 2024. Mudjacking: patching backdoor vulnerabilities in foundation models. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, Philadelphia, PA, pp. 2919–2936. https://www.usenix.org/conference/usenixsecurity24/presentation/liu-hongbin.

Loughlin, K., Rosenblum, J., Saroiu, S., Wolman, A., Skarlatos, D., Kasikci, B., 2023. Siloz: leveraging DRAM isolation domains to prevent inter-VM rowhammer. In: Proceedings of the 29th Symposium on Operating Systems Principles. Association for Computing Machinery, New York, NY, USA, p. 417–433. https://doi.org/10.1145/3600006.3613143

Lu, H., Wang, S., Wu, Y., He, W., Zhang, F., 2024. MOAT: towards safe BPF kernel extension. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, Philadelphia, PA, pp. 1153–1170. https://www.usenix.org/conference/usenixsecurity24/presentation/lu-hongyi.

Mccarty, B., 2005. SELinux NSA's Open Source Security Enhanced Linux. SELinux: NSA's Open Source Security Enhanced Linux.

Milajerdi, S.M., Eshete, B., Gjomemo, R., Venkatakrishnan, V.N., 2019a. Poirot: aligning attack behavior with kernel audit records for cyber threat hunting. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, p. 1795–1812. https://doi.org/10.1145/3319535.3363217

Milajerdi, S.M., Gjomemo, R., Eshete, B., Sekar, R., Venkatakrishnan, V.N., 2019b. Holmes: real-time APT detection through correlation of suspicious information flows. In: 2019 IEEE Symposium on Security and Privacy (SP), pp. 1137–1152. https://doi.org/10.1109/SP.2019.00026

Milburn, A., Van Der Kouwe, E., Giuffrida, C., 2022. Mitigating information leakage vulnerabilities with type-based data isolation. In: 2022 IEEE Symposium on Security and Privacy (SP), pp. 1049–1065. https://doi.org/10.1109/SP46214.2022.9833675

Narayan, S., Garfinkel, T., Taram, M., Rudek, J., Moghimi, D., Johnson, E., Fallin, C., Vahldiek-Oberwagner, A., LeMay, M., Sahita, R., Tullsen, D., Stefan, D., 2023. Going beyond the limits of SFI: flexible and secure hardware-assisted in-process isolation with HFI. In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. Association for Computing Machinery, New York, NY, USA, p. 266–281. https://doi.org/10.1145/3582016.3582023

Narayanan, V., Huang, Y., Tan, G., Jaeger, T., Burtsev, A., 2020. Lightweight kernel isolation with virtualization and VM functions. In: Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments. Association for Computing Machinery, New York, NY, USA, p. 157–171. https://doi.org/10.1145/3381052.3381328

Node2Vec, 2024. Node2Vec. node2vec: scalable feature learning for networks. https://snap.stanford.edu/node2vec/.

OWASP, 2024. Open web application security project. OWASP Top 10 vulnerabilities. https://owasp.org/www-project-top-ten/.

Pan, S., Tao, Z., Hoang, T., Zhang, D., Li, T., Xing, Z., Xu, X., Staples, M., Rakotoarivelo, T., Lo, D., 2024. A NEW HOPE: contextual privacy policies for mobile applications and an approach toward automated generation. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, Philadelphia, PA, pp. 5699–5716. https://www.usenix.org/conference/usenixsecurity24/presentation/pan-shidong-hope.

Pasquier, T., Han, X., Goldstein, M., Moyer, T., Eyers, D., Seltzer, M., Bacon, J., 2017. Practical whole-system provenance capture. In: Proceedings of the 2017 Symposium on Cloud Computing. Association for Computing Machinery, New York, NY, USA, p. 405–418. https://doi.org/10.1145/3127479.3129249

Pickren, R., Shekari, T., Zonouz, S.A., Beyah, R., 2024. Compromising industrial processes using web-based programmable logic controller malware. In: 31st Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26-March 1, 2024. The Internet Society. https://www.ndss-symposium.org/ndss-paper/compromising-industrial-processes-using-web-based-programmable-logic-controller-malware/.

Shao, M., Ding, Y., 2024. FVD-DPM: fine-grained vulnerability detection via conditional diffusion probabilistic models. In: Balzarotti, D., Xu, W. (Eds.), 33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14–16, 2024. USENIX Association. https://www.usenix.org/conference/usenixsecurity24/presentation/shao.

Shen, B., Shan, T., Zhou, Y., 2023. Multiview: finding blind spots in access-deny issues diagnosis. In: 32nd USENIX Security Symposium (USENIX Security 23). USENIX Association, Anaheim, CA, pp. 7499–7516. https://www.usenix.org/conference/usenixsecurity23/presentation/shen-bingyu-multiview.

SKlearn, 2024. SKlearn. scikit-learn machine learning in python. https://scikit-learn.org/stable/.

Skopik, F., Wurzenberger, M., Höld, G., Landauer, M., Kuhn, W., 2023. Behavior-based anomaly detection in log data of physical access control systems. IEEE Trans. Dependable Secur. Comput. 20 (4), 3158–3175. https://doi.org/10.1109/TDSC.2022.3197265

Snow, K.Z., Monrose, F., Davi, L., Dmitrienko, A., Liebchen, C., Sadeghi, A., 2013. Just-in-time code reuse: on the effectiveness of fine-grained address space layout randomization. In: 2013 IEEE Symposium on Security and Privacy, SP 2013, Berkeley, CA, USA, May 19–22, 2013. IEEE Computer Society, pp. 574–588. https://doi.org/10.1109/SP.2013.45

Tan, J., Jin, H., Hu, H., Hu, R., Zhang, H., Zhang, H., 2023a. WF-MTD: evolutionary decision method for moving target defense based on wright-fisher process. IEEE Trans. Dependable Secur. Comput. 20 (6), 4719–4732. https://doi.org/10.1109/TDSC.2022.3232537

Tan, J., Jin, H., Hu, H., Hu, R., Zhang, H., Zhang, H., 2023b. WF-MTD: evolutionary decision method for moving target defense based on wright-fisher process. IEEE Trans. Dependable Secur. Comput. 20 (6), 4719–4732. https://doi.org/10.1109/TDSC.2022.3232537

Tchimwa Bouom, A. L.P., Lienou, J.-P., Ejuh Geh, W., Free Nelson, F., Shetty, S., Kamhoua, C., 2024. TriAssetRank: ranking vulnerabilities, exploits, and privileges for countermeasures prioritization. IEEE Trans. Inf. Forensics Secur. 19, 10189–10205. https://doi.org/10.1109/TIFS.2024.3488533

Tian, D.J., Hernandez, G., Choi, J.I., Frost, V., Johnson, P.C., Butler, K. R.B., 2019. LBM: a security framework for peripherals within the linux kernel. In: 2019 IEEE

Symposium on Security and Privacy (SP), pp. 967–984. https://doi.org/10.1109/SP.2019.00041

Verizon, 2024. Verizon. 2024 data breach investigations report: vulnerability exploitation boom threatens cybersecurity. https://www.verizon.com/about/news/2024-data-breach-investigations-report-vulnerability-exploitation-boom.

VirusShare, 2024. VirusShare. https://virusshare.com/.

VirusTotal, 2024. VirusTotal. https://www.virustotal.com/gui/home/upload.

Wang, Q., Hassan, W.U., Li, D., Jee, K., Yu, X., Zou, K., Rhee, J., Chen, Z., Cheng, W., Gunter, C.A., Chen, H., 2020. You are what you do: hunting stealthy malware via data provenance analysis. In: 27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23–26, 2020. The Internet Society. https://www.ndss-symposium.org/ndss-paper/you-are-what-you-do-hunting-stealthy-malware-via-data-provenance-analysis/.

Wu, C., Xie, M., Wang, Z., Zhang, Y., Lu, K., Zhang, X., Lai, Y., Kang, Y., Yang, M., Li, T., 2023. Dancing with wolves: an intra-process isolation technique with privileged hardware. IEEE Trans. Dependable Secur. Comput. 20 (3), 1959–1978. https://doi.org/10.1109/TDSC.2022.3168089

Xiao, L., Wang, H., Yu, A., Zhao, L., Meng, D., 2025. CASPR: context-aware security policy recommendation. In: 32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24–28, 2025. The Internet Society. https://www.ndss-symposium.org/ndss-paper/caspr-context-aware-security-policy-recommendation/.

Xie, M., Wu, C., Zhang, Y., Xu, J., Lai, Y., Kang, Y., Wang, W., Wang, Z., 2022. CETIS: retrofitting intel cet for generic and efficient intra-process memory isolation. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, p. 2989–3002. https://doi.org/10.1145/3548606.3559344

Xing, Y., Wang, X., Torabi, S., Zhang, Z., Lei, L., Sun, K., 2024. A hybrid system call profiling approach for container protection. IEEE Trans. Dependable Secur. Comput. 21 (3), 1068–1083. https://doi.org/10.1109/TDSC.2023.3268124

Xu, J., Xie, M., Wu, C., Zhang, Y., Li, Q., Huang, X., Lai, Y., Kang, Y., Wang, W., Wei, Q., Wang, Z., 2023. Panic: pan-assisted intra-process memory isolation on arm. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, p. 919–933. https://doi.org/10.1145/3576915.3623206

Xu, Z., Fang, P., Liu, C., Xiao, X., Wen, Y., Meng, D., 2022. DepComm: graph summarization on system audit logs for attack investigation. In: 2022 IEEE Symposium on Security and Privacy (SP), pp. 540–557. https://doi.org/10.1109/SP46214.2022.9833632

Yang, F., Im, B., Huang, W., Kaoudis, K., Vahldiek-Oberwagner, A., Tsai, C.-c., Dautenhahn, N., 2024. Endokernel: a thread safe monitor for lightweight subprocess isolation. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, Philadelphia, PA, pp. 145–162. https://www.usenix.org/conference/usenixsecurity24/presentation/yang-fangfei.

Yang, Y., Zhang, Y., Lin, Z., 2022. Cross miniapp request forgery: root causes, attacks, and vulnerability detection. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, p. 3079–3092. https://doi.org/10.1145/3548606.3560597

Yu, D., Yang, G., Meng, G., Gong, X., Zhang, X., Xiang, X., Wang, X., Jiang, Y., Chen, K., Zou, W., Lee, W., Shi, W., 2021. Sepal: towards a large-scale analysis of seandroid policy customization. In: Proceedings of the Web Conference 2021. Association for Computing Machinery, New York, NY, USA, p. 2733–2744. https://doi.org/10.1145/3442381.3450007

Yuan, S., Besson, F., Talpin, J.-P., Hym, S., Zandberg, K., Baccelli, E., 2022. End-to-end mechanized proof of an eBPF virtual machine for micro-controllers. In: Shoham, S., Vizel, Y. (Eds.), Computer Aided Verification. Springer International Publishing, Cham, pp. 293–316.

Zeng, J., Chua, Z.L., Chen, Y., Ji, K., Liang, Z., Mao, J., 2021. WATSON: abstracting behaviors from audit logs via aggregation of contextual semantics. In: 28th Annual Network and Distributed System Security Symposium, NDSS 2021, Virtually, February 21–25, 2021. The Internet Society. https://www.ndss-symposium.org/ndss-paper/watson-abstracting-behaviors-from-audit-logs-via-aggregation-of-contextual-semantics/.

Zhang, N., Li, Q., 2023. MTD '23: 10th acm workshop on moving target defense. In: Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. Association for Computing Machinery, New York, NY, USA, p. 3653–3654. https://doi.org/10.1145/3576915.3624022

Zhang, P., Wu, C., Meng, X., Zhang, Y., Peng, M., Zhang, S., Hu, B., Xie, M., Lai, Y., Kang, Y., Wang, Z., 2024. HIVE: a hardware-assisted isolated execution environment for eBPF on AArch64. In: 33rd USENIX Security Symposium (USENIX Security 24). USENIX Association, Philadelphia, PA, pp. 163–180. https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-peihua.

Zhang, Z., Johnson, C., Venkatasubramanian, N., Ren, S., 2022. Process scenario discovery from event logs based on activity and timing information. J. Syst. Archit. 125, 102435. https://doi.org/https://doi.org/10.1016/j.sysarc.2022.102435

Zhao, B., Li, Z., Qin, S., Ma, Z., Yuan, M., Zhu, W., Tian, Z., Zhang, C., 2022. StateFuzz: system call-based state-aware linux driver fuzzing. In: 31st USENIX Security Symposium (USENIX Security 22). USENIX Association, Boston, MA, pp. 3273–3289. https://www.usenix.org/conference/usenixsecurity22/presentation/zhao-bodong.

Zhong, Y., Li, H., Wu, Y.J., Zarkadas, I., Tao, J., Mesterhazy, E., Makris, M., Yang, J., Tai, A., Stutsman, R., Cidon, A., 2022. XRP: In-Kernel storage functions with eBPF. In: 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22). USENIX Association, Carlsbad, CA, pp. 375–393. https://www.usenix.org/conference/osdi22/presentation/zhong.

Zhou, Y., Xiang, X., Kiley, M., Dharanipragada, S., Yu, M., 2024. DINT: fast in-kernel distributed transactions with eBPF. In: 21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24). USENIX Association, Santa Clara, CA, pp. 401–417. https://www.usenix.org/conference/nsdi24/presentation/zhou-yang.

Zhu, T., Yu, J., Xiong, C., Cheng, W., Yuan, Q., Ying, J., Chen, T., Zhang, J., Lv, M., Chen, Y., Wang, T., Fan, Y., 2023. APTSHIELD: a stable, efficient and real-time apt detection system for linux hosts. IEEE Trans. Dependable Secur. Comput. 20 (6), 5247–5264. https://doi.org/10.1109/TDSC.2023.3243667

Zlib, 2024. Zlib. https://www.zlib.net/.